

"""

certifier.py – GRUS Audit Pipeline Stage 10

Specification: GRUS-STD-001-v1.0 Section 3 Stage 10; GRUS-SEAL-001-v1.0

Signs the completed certification record using the LLC's ed25519 private key and produces the validated-fact stamp.

KEY HANDLING:

- Production: reads the LLC's persistent private key from \$GRUS_PRIVATE_KEY_PATH (default: /etc/grus/private_key.pem). The key MUST exist before Stage 10 runs in production. Generate the key once via generate_grus_keys.py.
- Development / contributor forks / CI testing: if --allow-ephemeral-key is passed AND no private key is found, the certifier generates a temporary keypair, signs the record, and writes the ephemeral public key alongside the certification so verification is possible within the test session. Ephemeral seals carry the prefix "ed25519-EPHEMERAL:" and are NOT GRUS-CERTIFIED – they exist only for fork testing and CI dry runs.

Published by: Green Recursive Utility Service LLC

"""

```
import os
import sys
import base64
import hashlib
import yaml
from typing import Dict, Any, Tuple
from dataclasses import dataclass

try:
    from cryptography.hazmat.primitives.asymmetric.ed25519 import (
        Ed25519PrivateKey, Ed25519PublicKey)
    from cryptography.hazmat.primitives import serialization
    HAVE_CRYPTO = True
except ImportError:
    HAVE_CRYPTO = False

GRUS_PRIVATE_KEY_PATH = os.environ.get(
    "GRUS_PRIVATE_KEY_PATH", "/etc/grus/private_key.pem"
)
GRUS_PUBLIC_KEY_PATH = os.environ.get(
    "GRUS_PUBLIC_KEY_PATH", "/etc/grus/public_key.pem"
)

@dataclass
```

```

class CertificationResult:
    passed: bool
    audit_id: str = ""
    grus_seal: str = ""
    record: Dict[str, Any] = None
    failure_reason: str = ""
    is_ephemeral: bool = False

def canonicalize_yaml(record: Dict[str, Any]) -> bytes:
    """Canonical YAML form excluding the seal field, deterministically ordered."""
    record_no_seal = {k: v for k, v in record.items() if k != "grus_seal"}
    return yaml.safe_dump(
        record_no_seal, sort_keys=True, default_flow_style=False
    ).encode("utf-8")

def load_or_generate_key(
    private_key_path: str,
    allow_ephemeral: bool = False,
) -> Tuple[Ed25519PrivateKey, bool]:
    """
    Load the LLC's persistent private key, or generate an ephemeral one
    if allow_ephemeral=True and no persistent key exists.

    Returns:
        (private_key, is_ephemeral)

    Raises:
        FileNotFoundError if no key found and allow_ephemeral is False
    """
    if not HAVE_CRYPTO:
        raise RuntimeError(
            "cryptography library not installed. Install: pip install cryptography"
        )

    if os.path.exists(private_key_path):
        with open(private_key_path, "rb") as f:
            priv = serialization.load_pem_private_key(f.read(), password=None)
            return priv, False

    if not allow_ephemeral:
        raise FileNotFoundError(
            f"GRUS private key not found at {private_key_path}. "
            f"Generate via: python3 generate_grus_keys.py --output-dir "
            f"{os.path.dirname(private_key_path) or '/etc/grus'}"
        )

    # Ephemeral fallback for dev/CI/fork testing
    priv = Ed25519PrivateKey.generate()

```

```

return priv, True

def sign_record(
    record: Dict[str, Any],
    private_key_path: str = None,
    allow_ephemeral: bool = False,
) -> Tuple[str, bool]:
    """
    Sign the certification record. Returns (seal_string, is_ephemeral).

    The seal_string is one of:
        "ed25519:<base64>"          (production, GRUS-certified)
        "ed25519-EPHEMERAL:<base64>" (dev/CI, NOT GRUS-certified)
    """
    private_key_path = private_key_path or GRUS_PRIVATE_KEY_PATH
    priv, ephemeral = load_or_generate_key(private_key_path, allow_ephemeral)

    canonical = canonicalize_yaml(record)
    canonical_hash = hashlib.sha256(canonical).digest()
    seal_bytes = priv.sign(canonical_hash)
    encoded = base64.b64encode(seal_bytes).decode("ascii")

    prefix = "ed25519-EPHEMERAL" if ephemeral else "ed25519"
    return f"{prefix}:{encoded}", ephemeral

def verify_seal(record: Dict[str, Any], public_key_path: str = None) -> bool:
    """
    Verify a sealed record against the LLC's public key.
    Returns True if verification passes, False otherwise.
    """
    public_key_path = public_key_path or GRUS_PUBLIC_KEY_PATH
    if not HAVE_CRYPTO:
        return False
    seal_str = record.get("grus_seal", "")
    if not seal_str:
        return False

    if seal_str.startswith("ed25519-EPHEMERAL:"):
        # Ephemeral seals never verify against the canonical public key
        # – they are not GRUS-certified by design.
        return False
    if not seal_str.startswith("ed25519:"):
        return False

    seal_bytes = base64.b64decode(seal_str[len("ed25519:"):])

    if not os.path.exists(public_key_path):
        return False

```

```

with open(public_key_path, "rb") as f:
    pub = serialization.load_pem_public_key(f.read())

canonical = canonicalize_yaml(record)
canonical_hash = hashlib.sha256(canonical).digest()
try:
    pub.verify(seal_bytes, canonical_hash)
    return True
except Exception:
    return False

def issue_certification(
    stage_results: Dict,
    audit_id: str,
    private_key_path: str = None,
    allow_ephemeral: bool = False,
) -> CertificationResult:
    """
    Build the certification record and sign it.

    On production deployment (canonical LLC pipeline): allow_ephemeral=False;
    requires the persistent private key to exist.

    On dev/CI/contributor-fork runs: allow_ephemeral=True; the resulting seal
    is marked "ed25519-EPHEMERAL:" so it cannot be confused with a real
    GRUS-certified seal.
    """
    record = {
        "audit_id": audit_id,
        "audit_date": stage_results.get("audit_date", ""),
        "audit_pipeline_version": "1.0",
        "submission": stage_results.get("submission", {}),
        "manifest_hash": stage_results.get("hashing", {}).get("manifest_hash", ""),
        "verify_py_hash": stage_results.get("hashing", {}).get("verify_py_hash", ""),
        "constants_py_hash": stage_results.get("hashing", {}).get("constants_py_hash", ""),
        "datasets": stage_results.get("datasets", []),
        "residuals_matched": stage_results.get("residual_comparison", {}).get("matched", []),
        "closure": stage_results.get("closure", {}),
        "falsification_validation": stage_results.get("falsification", {}),
        "clauses": {
            "clause_1_zero_free_parameters": "pass",
            "clause_2_zero_placeholders": "pass",
            "clause_3_public_data_reproducibility": "pass",
            "clause_4_overconstrained_closure": "pass",
            "clause_5_forward_facing_falsification": "pass",
            "clause_6_immutable_public_timestamping": "pass",
        },
        "audit_result": "VALIDATED FACT",
        "issued_by": "Green Recursive Utility Service LLC",
    }

```

[illegible]

```
allow_ephemeral=args.allow_ephemeral)

if r.passed:
    kind = "EPHEMERAL (dev/test)" if r.is_ephemeral else "CANONICAL (GRUS-certified)"
    print(f"[ISSUED] {r.audit_id} – {kind}")
    print(f"  Seal: {r.grus_seal[:80]}...")
    print(f"  Result: {r.record['audit_result']}")
else:
    print(f"[FAIL] {r.failure_reason}")
    sys.exit(1)
```